C SKILL LAB III - NODE.JS LAB MANUAL

Practical 01: Introduction to Node.js

AIM:
To Install Node.js and write a basic program to print 'Hello from Node.js!' to console.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Node.js is a server-side platform built on Google Chrome's JavaScript Engine (V8 Engine). It is used to build scalable network applications using JavaScript. The 'console.log()' function is used to print messages to the console, and it serves as the simplest way to output information in Node.js.

CODE:
```
console.log("Hello from Node.js!");
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 02: Using Node.js as a Web Server

AIM:
To Create a basic HTTP server that sends a welcome message to the browser.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Node.js provides the built-in 'http' module to create web servers. The server listens on a port and responds to HTTP requests using the 'createServer' method.

CODE:

```
const http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Welcome to Node.js Web Server!');
}).listen(3000);
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 03: Working with Built-in Modules

AIM:
To Demonstrate the use of fs and os modules in Node.js.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Node.js comes with many built-in modules. The 'fs' module is used for file system operations like reading and writing files. The 'os' module provides information about the operating system.

CODE:

```
const fs = require('fs');
const os = require('os');

fs.writeFileSync('sample.txt', 'This file is created using fs module.');
console.log('System Info:', os.platform(), os.release());
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 04: Creating Custom Modules

AIM:
To Create and use a custom module that performs a mathematical operation.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
In Node.js, modules are reusable blocks of code. Custom modules can be created by exporting functions using 'module.exports'.

CODE:

```
// mathModule.js
exports.add = function(a, b) {
   return a + b;
};


// main.js
const math = require('./mathModule');
console.log("Addition:", math.add(10, 5));
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 05: Handling URLs and File System

AIM:
To Parse a URL and return a file's content. Handle 404 errors gracefully.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
The 'url' module is used to parse URLs, and 'fs' is used to read files. Together, they can serve different files based on the request path.

CODE:

```
const http = require('http');
const fs = require('fs');
const url = require('url');

http.createServer((req, res) => {
  const q = url.parse(req.url, true);
  const filename = "." + q.pathname;
  fs.readFile(filename, (err, data) => {
    if (err) {
      res.writeHead(404, {'Content-Type': 'text/html'});
      return res.end("404 Not Found");
    }
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write(data);
    return res.end();
  });
}).listen(3000);
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?

3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 06: Using External NPM Packages

AIM:
To Install and use the upper-case NPM package to transform text.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
'npm install' is used to install third-party packages. The 'upper-case' package converts strings to uppercase.

CODE:

```
// Run: npm install upper-case
const uc = require('upper-case');
console.log(uc.upperCase("hello world"));
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 07: Event Handling in Node.js

AIM:
To Use the events module and EventEmitter to demonstrate event handling.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Node.js uses the EventEmitter class for event-driven programming. Custom events can be defined and listened to.

CODE:

```
const events = require('events');
const eventEmitter = new events.EventEmitter();

eventEmitter.on('greet', () => {
  console.log('Hello there!');
});

eventEmitter.emit('greet');
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 08: Handling File Uploads

AIM:
To Use the formidable module to upload files to the server.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Formidable is a Node.js module for parsing form data, especially file uploads.

CODE:

```
// Run: npm install formidable
const http = require('http');
const formidable = require('formidable');
const fs = require('fs');

http.createServer((req, res) => {
 if (req.url == '/fileupload') {
   const form = new formidable.IncomingForm();
   form.parse(req, (err, fields, files) => {
     const oldpath = files.filetoupload.filepath;
     const newpath = './' + files.filetoupload.originalFilename;
     fs.rename(oldpath, newpath, (err) => {
       if (err) throw err;
       res.write('File uploaded and moved!');
       res.end();
     });
   });
 } else {
   res.write('<form action="fileupload" method="post" enctype="multipart/form-data">');
   res.write('<input type="file" name="filetoupload"><br>');
   res.write('<input type="submit">');
   res.write('</form>');
   return res.end();
 }
}).listen(3000);
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 09: Sending Emails with Node.js

AIM:
To Use the nodemailer module to send emails from a Node.js server.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Nodemailer is a module for sending emails using Node.js and SMTP credentials.

CODE:

```
// Run: npm install nodemailer
const nodemailer = require('nodemailer');

let transporter = nodemailer.createTransport({
 service: 'gmail',
 auth: {
  user: 'your-email@gmail.com',
  pass: 'your-password'
 }
});

let mailOptions = {
 from: 'your-email@gmail.com',
 to: 'receiver@example.com',
 subject: 'Test Email',
 text: 'Hello from Node.js!'
};

transporter.sendMail(mailOptions, (error, info) => {
 if (error) {
  return console.log(error);
 }
 console.log('Email sent: ' + info.response);
});
```

CONCLUSION:

The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:

1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 10: MySQL Database Connection

AIM:
To Install MySQL driver, establish a connection using Node.js.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
The 'mysql' module enables MySQL connectivity in Node.js. After installing, connections can be made using credentials.

CODE:

```
// Run: npm install mysql
const mysql = require('mysql');

const con = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: ""
});

con.connect((err) => {
  if (err) throw err;
  console.log("Connected to MySQL!");
});
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 11: Creating a Database and Table

AIM:
To Create a MySQL database and a table using Node.js code.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Once connected to MySQL, 'CREATE DATABASE' and 'CREATE TABLE' statements can be executed using the query method.

CODE:

```
const mysql = require('mysql');
const con = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: ""
});

con.connect(err => {
  if (err) throw err;
  con.query("CREATE DATABASE testdb", (err, result) => {
    if (err) throw err;
    console.log("Database created");
  });
});
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 12: Inserting Records into MySQL

AIM:
To Insert single and multiple rows into a MySQL table via Node.js.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Data insertion in MySQL is done using the INSERT INTO statement. Node.js sends these queries through the MySQL module.

CODE:

```
const mysql = require('mysql');
const con = mysql.createConnection({
 host: "localhost",
 user: "root",
 password: "",
 database: "testdb"
});

con.connect(err => {
 if (err) throw err;
 const sql = "INSERT INTO users (name, email) VALUES ?";
 const values = [
  ['John Doe', 'john@example.com'],
  ['Jane Smith', 'jane@example.com']
 ];
 con.query(sql, [values], (err, result) => {
  if (err) throw err;
  console.log("Records inserted: " + result.affectedRows);
 });
});
```

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:

1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 13: Email Sender App

AIM:
To Build a functional email sending app using nodemailer with user input.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Using user input from a web form, nodemailer can be configured to send custom messages.

CODE:
// Similar setup as nodemailer example, include express and body-parser for form input.

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?

Practical 14: Basic User Management Web App

AIM:
To Create a simple web app for user sign-up/login and profile viewing using Express.js and MySQL.

OBJECTIVES:
- Understand the core concept of the topic.
- Apply Node.js syntax and structure.
- Develop a basic program as described.

SCOPE:
This practical will enable students to apply foundational concepts of Node.js development in real-world backend/server scenarios.

THEORY:
Express.js is used to create routes and handle HTTP requests, while MySQL stores user credentials and profiles.

CODE:
// Setup requires express, mysql, body-parser. Create routes for /register, /login, /profile.

CONCLUSION:
The practical was successfully executed, demonstrating the use of Node.js features as per the objective.

VIVA QUESTIONS:
1. What is the purpose of this practical?
2. Which Node.js feature/module was used?
3. Explain the syntax used in the program.
4. What real-life applications use similar concepts?